Rumbo

Aplicación de gestión de tareas — Informe de implementación

Vue 3 + Vuetify · Node.js + Express + TypeScript · SQL Server 2022 · Docker Compose

Fecha: 1 de octubre de 2026

Ejecución: `docker compose up --build` — se levanta todo, incluida la base de datos.

Acceso: http://localhost:8080 · usuario **admin** · contraseña **admin123**

Contenido

1. Resumen y cumplimiento de requisitos
2. Base de datos: tabla, índice y procedimientos
3. Backend: API, seguridad y validación
4. Frontend: interfaz, filtros y panel de análisis
5. Funcionalidades extra (búsqueda, fechas, orden, culminación)
6. Docker, calidad y pruebas
7. Preguntas de la prueba

1. Resumen y cumplimiento

Rumbo es una aplicación de tareas lista para ejecutarse con un solo comando. Cada punto de la prueba está implementado y verificado en vivo: las capturas de este informe salen del sistema en funcionamiento.

Requisito	Cómo se cumple	Estado
Levantar con un comando	`docker compose up --build` crea base, backend y frontend en orden	✓
Tabla y scripts SQL	Tabla Tasks y `init.sql` en el repositorio	✓
Stored procedures con TRY/CATCH y transacción	sp_GetTasks, sp_CreateTask, sp_UpdateTaskStatus, sp_GetTaskStats	✓
Filtro por estado y paginación en SQL	Resueltos dentro de sp_GetTasks con OFFSET/FETCH	✓
Índice que optimiza una consulta real	IX_Tasks_Status_CreatedAt (verificado en el plan)	✓
API REST listar/crear/cambiar estado	Endpoints bajo /api/tasks	✓
Login con JWT que protege endpoints	/api/auth/login + middleware Bearer	✓
Validación, errores centralizados, códigos HTTP	Zod + middleware de errores uniforme	✓
Consultas parametrizadas y variables de entorno	Parámetros tipados en los SP; configuración por entorno	✓
Frontend con carga, error y composable propio	Vue 3 + Vuetify; composable useTasks	✓
Preguntas respondidas en el README	Sección dedicada en README.md	✓

Verificación en vivo

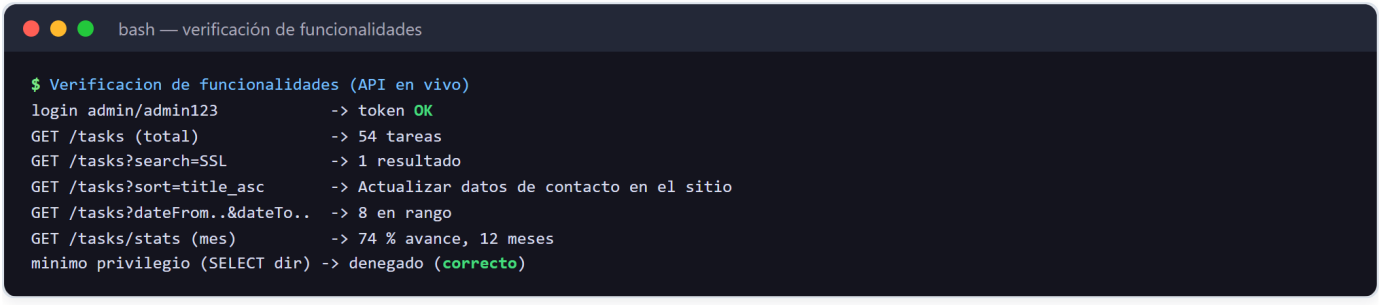


Figura 1. Funcionalidades comprobadas contra la API en ejecución.

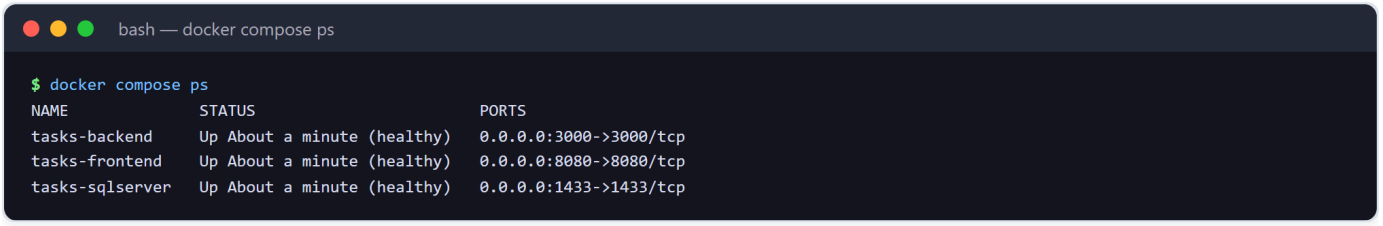


Figura 2. Los tres servicios activos y saludables tras levantar el stack.

Más allá de lo pedido

Identidad propia (marca Rumbo), modo claro y oscuro, buscador, filtro por rango de fechas con calendario, orden alfabético, fecha de culminación, panel de análisis por semana/mes/año, pruebas automáticas, ESLint/Prettier, integración continua y una variante de despliegue endurecida.

2. Base de datos

La base se crea sola al levantar el stack: un contenedor de inicialización espera a SQL Server y ejecuta `init.sql`. El script es idempotente, así que puede correr en cada arranque sin duplicar datos.

Tabla Tasks

- `Id` clave primaria autoincremental.
- `Title` (hasta 150), `Description` opcional.
- `Status`: PENDING o COMPLETED (por defecto PENDING).
- `CreatedAt` fecha de creación y `CompletedAt` fecha de culminación (vacía mientras está pendiente).
- Restricciones CHECK sobre el estado y sobre el título no vacío.

Índice y procedimientos

bash — objetos de la base

```
$ sqlcmd: objetos de la base (tabla + stored procedures) e indice
Objeto Tipo
------ ----
Tasks USER_TABLE
sp_CreateTask SQL_STORED_PROCEDURE
sp_GetTasks SQL_STORED_PROCEDURE
sp_GetTaskStats SQL_STORED_PROCEDURE
sp_UpdateTaskStatus SQL_STORED_PROCEDURE
Indice Columnas
------ --------
IX_Tasks_Status_CreatedAt Status, CreatedAt DESC, Id DESC, Title (INCLUDE)
PK_Tasks Id
```

Figura 3. Tabla, cuatro stored procedures e índice compuesto con sus columnas.

Procedimiento	Qué hace
sp_GetTasks	Devuelve el total y la página. Filtra por estado, texto y rango de fechas; ordena por fecha o por título; pagina con OFFSET/FETCH.
sp_CreateTask	Inserta una tarea y devuelve el registro creado.
sp_UpdateTaskStatus	Cambia el estado, valida que exista y sella o limpia la fecha de culminación.
sp_GetTaskStats	Agrega métricas por semana, mes o año para el panel de análisis.

Todos usan `SET XACT_ABORT ON`, `BEGIN TRY/CATCH`, transacción explícita y `ROLLBACK` ante error. Los errores de negocio se lanzan con `THROW 50400` y `50404`, que el backend traduce a 400 y 404.

El índice y su plan

El índice `IX_Tasks_Status_CreatedAt (Status, CreatedAt DESC, Id DESC) INCLUDE (Title)` optimiza el listado: entrega las filas ya filtradas y ordenadas, evitando el paso de ordenar.

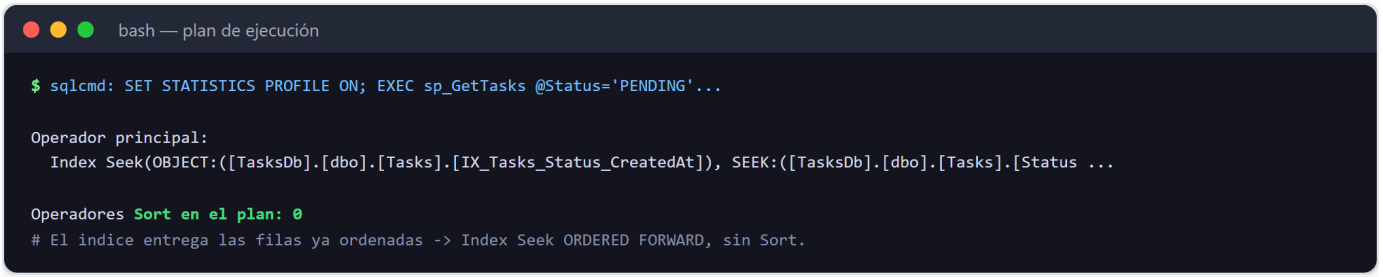


Figura 4. Plan de ejecución: Index Seek ordenado y cero operadores Sort.

3. Backend

Método	Ruta	Login	Qué hace
GET	/api/health	No	Estado de la API y de la base
POST	/api/auth/login	No	Devuelve un token JWT (limitado a 20 intentos cada 15 min)
GET	/api/tasks	Sí	Lista con filtros, búsqueda, orden y paginación
POST	/api/tasks	Sí	Crea una tarea
PATCH	/api/tasks/:id/status	Sí	Cambia el estado
GET	/api/tasks/stats	Sí	Métricas para el panel de análisis

Seguridad y validación

- JWT firmado con algoritmo fijo; las credenciales se comparan en tiempo constante.
- Validación estricta con Zod en body, query y params; los errores se devuelven juntos y con el mismo formato.
- Helmet, límite de tamaño del cuerpo y limitación de intentos de login.
- El backend se conecta con un usuario de base de datos que solo puede ejecutar los procedimientos, no leer las tablas.

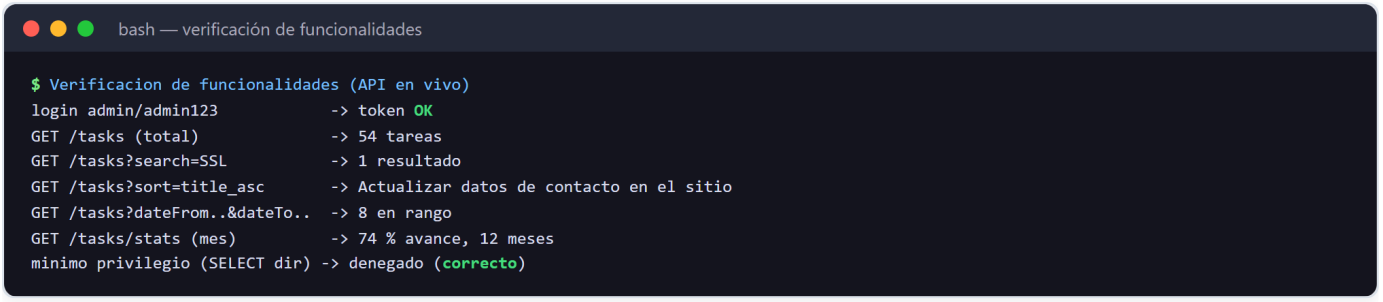


Figura 5. Respuestas de la API: login, filtros, estadísticas y rechazo de acceso directo a las tablas.

Arquitectura en capas: rutas, controladores, servicios, repositorios, middlewares y configuración. Todo tipado con TypeScript estricto.

4. Frontend

Interfaz en Vue 3 con Vuetify. Identidad propia (Rumbo), paleta azul sobria y modo claro y oscuro. Estados de carga con skeletons y errores visibles. La lógica de tareas vive en el composable `useTasks`.

Inicio de sesión

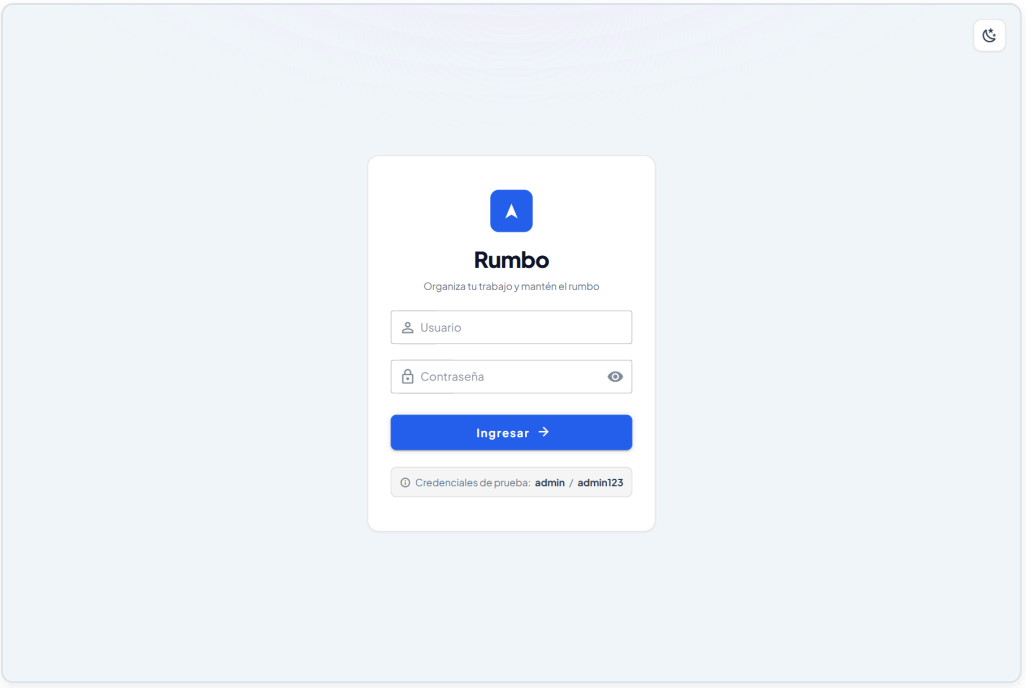


Figura 6. Pantalla de login.

Pantalla de tareas

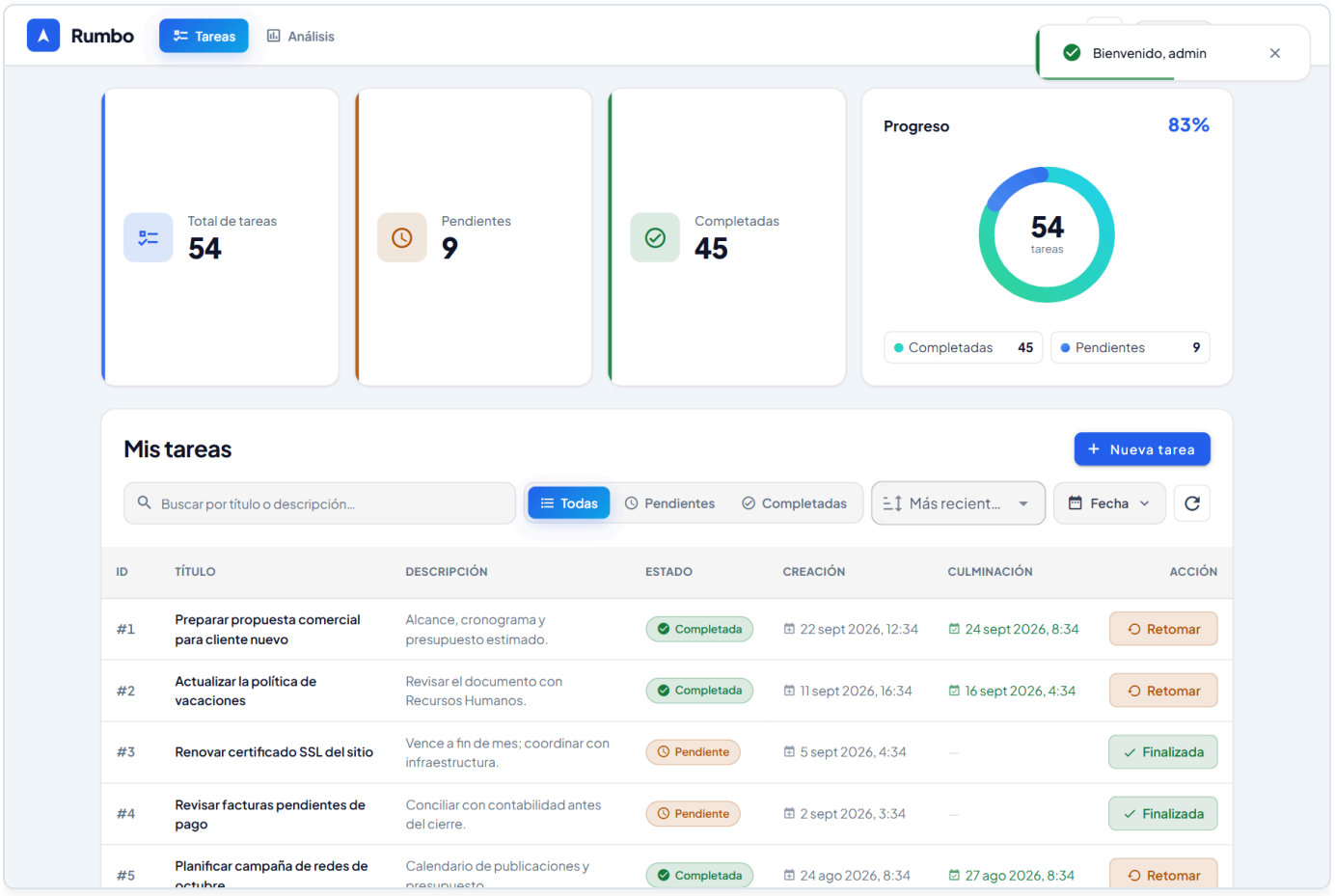


Figura 7. Tablero en modo claro: resumen, progreso, buscador, filtros y tabla.

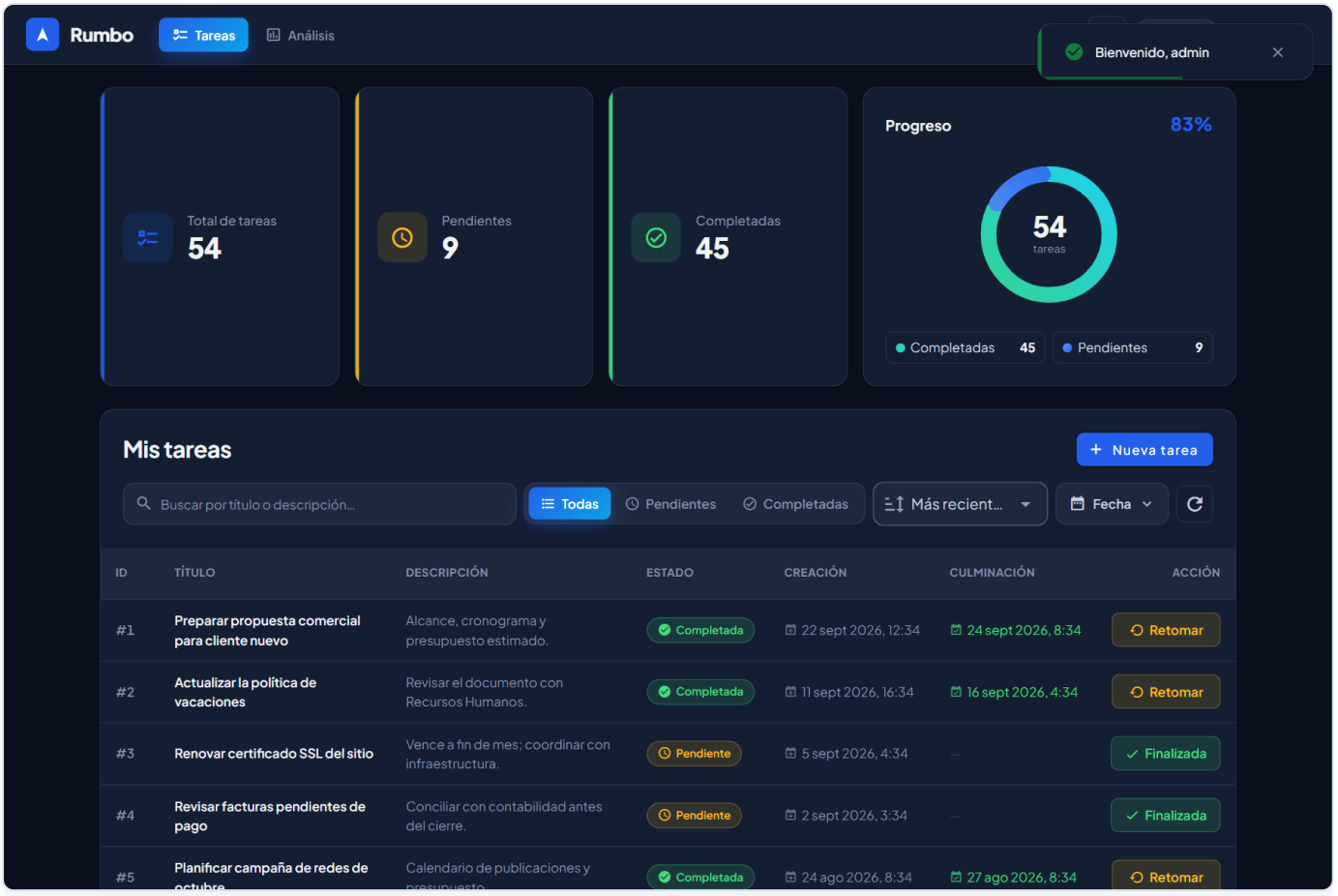


Figura 8. El mismo tablero en modo oscuro.

5. Funcionalidades extra

Buscador y orden

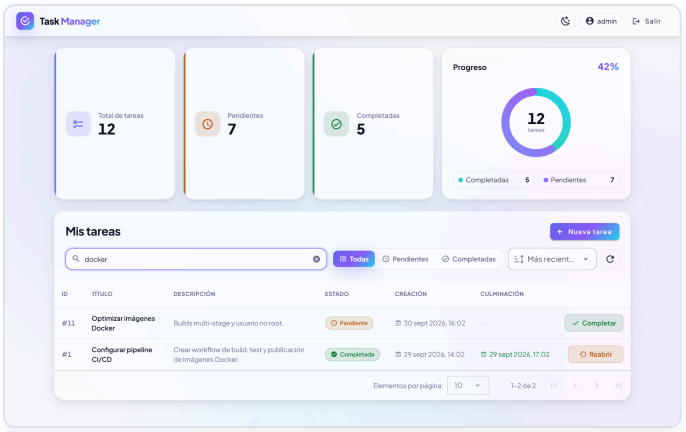


Figura 9. Búsqueda por texto resuelta en el servidor.

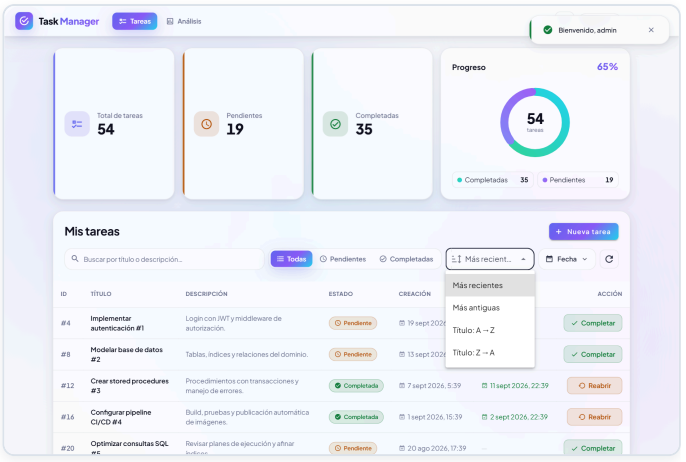


Figura 10. Orden por fecha o alfabético (A→Z, Z→A).

Filtro por fecha con calendario

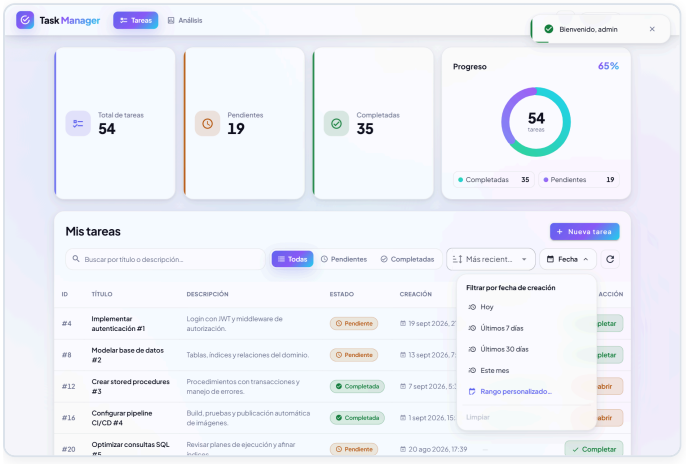


Figura 11. Menú de fecha con opciones rápidas.

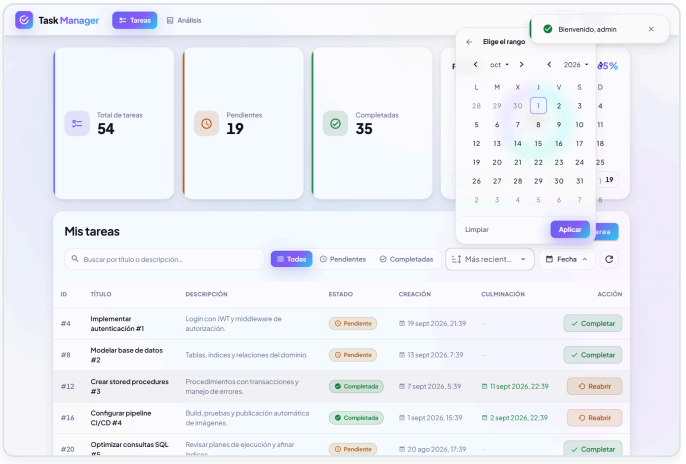


Figura 12. Calendario para elegir un rango personalizado.

Estado y fecha de culminación

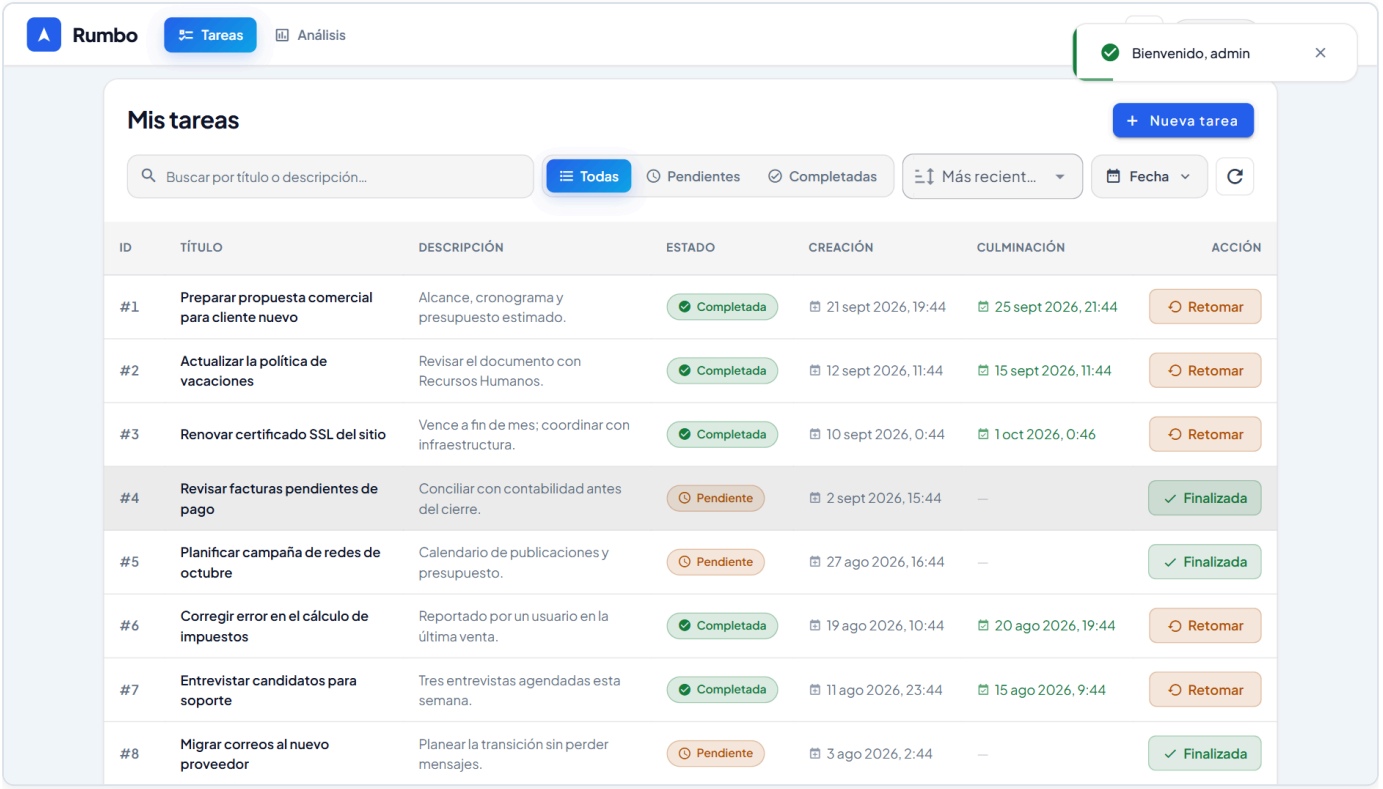


Figura 13. Columna de acción (Finalizada / Retomar) y fechas de creación y culminación.

Panel de análisis

Métricas y evolución de las tareas, con vista por semana, mes o año.

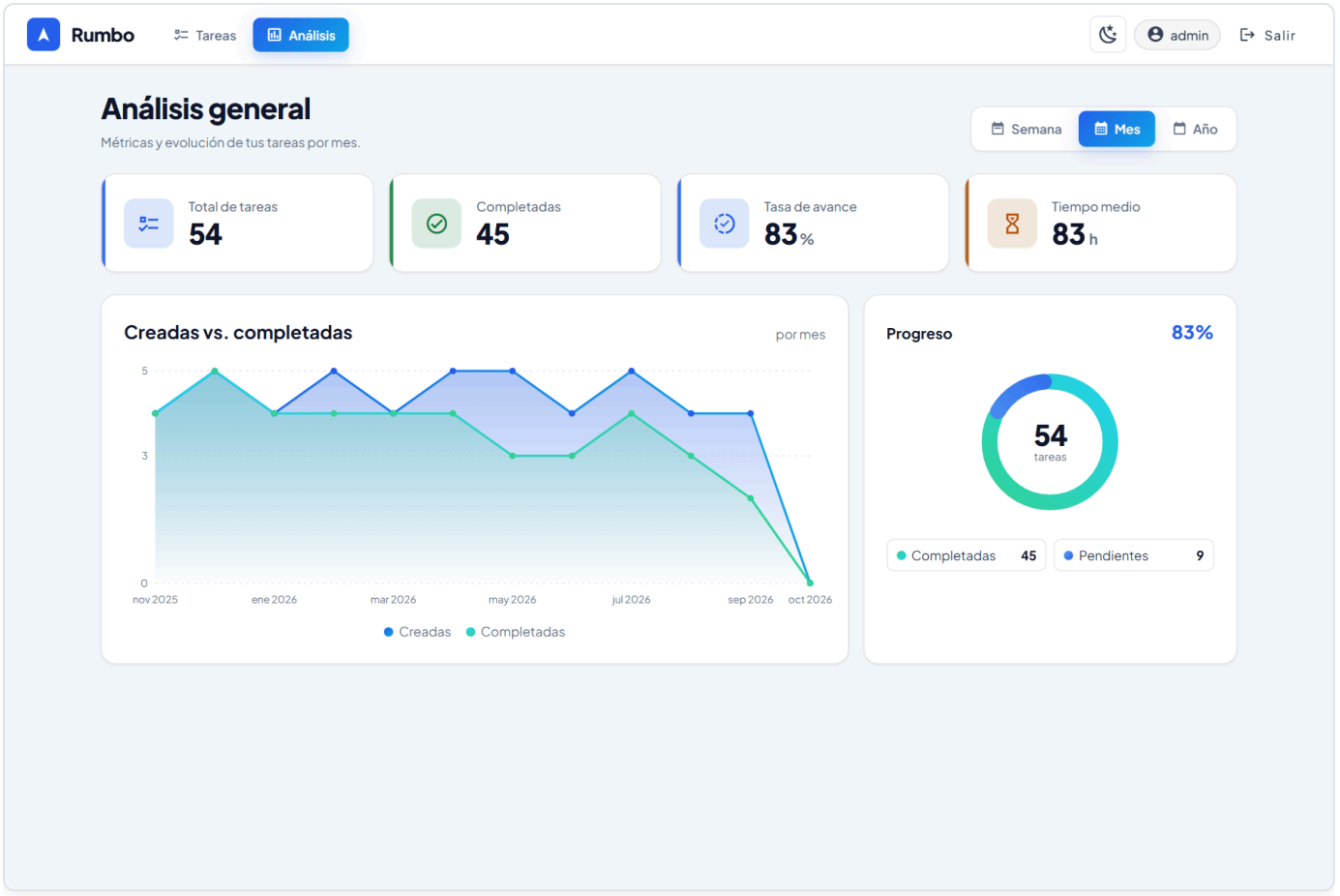


Figura 14. Panel de análisis: indicadores, gráfico de creadas contra completadas y progreso.

6. Docker, calidad y pruebas

Contenedores e imágenes

- Backend: imagen multi-stage con usuario sin privilegios.
- Frontend: build con Vite servido por nginx sin usuario root.
- Base de datos: SQL Server 2022 con un job de inicialización idempotente.
- Orden de arranque garantizado con healthchecks: base → inicialización → backend → frontend.

Pruebas automáticas

bash — pruebas automáticas

```
$ cd backend && npm test        (Vitest + Supertest)
 ✓ tests/api.test.ts (20 tests) 433ms
 Test Files  1 passed (1)
      Tests  20 passed (20)

$ cd frontend && npm test       (Vitest + Vue Test Utils)
 ✓ src/components/TaskStatusChip.test.ts (2 tests) 37ms
 ✓ src/composables/useTasks.test.ts (6 tests) 17ms
 Test Files  2 passed (2)
      Tests  8 passed (8)
```

Figura 15. 20 pruebas de backend y 8 de frontend, todas en verde.

Lint y despliegue de producción

bash — lint y producción

```
$ npm run lint   (backend y frontend)
backend  -> 0 errores, 0 warnings
frontend -> 0 errores, 0 warnings

$ docker compose config (prod) | grep published
frontend -> 8080   (backend y SQL Server no se publican: hallazgo H-4)
```

Figura 16. ESLint sin errores y variante de producción que no expone los puertos internos.

Además hay integración continua en `.github/workflows/ci.yml`: revisa el código, corre las pruebas, construye las imágenes, las escanea y hace una prueba de humo levantando el stack.

7. Preguntas de la prueba

¿Qué consulta optimiza el índice y cómo verificarlo?

Optimiza el listado principal (filtrar por estado, ordenar por fecha y paginar). Como el índice guarda las filas ya ordenadas, SQL Server las lee en orden y se ahorra el paso de ordenar. Se verifica mirando el plan de ejecución: debe aparecer un Index Seek sobre el índice y ningún operador Sort (ver figura 4).

Si creciera a 5 millones de tareas, ¿qué cambiarías?

- Paginación por clave en vez de OFFSET/FETCH, que se vuelve lento en páginas altas.
- Conteo aproximado en vez de contar el total exacto en cada consulta.
- Full-Text Search para la búsqueda por texto.
- Tabla de resumen o índice columnstore para el análisis.
- Particionado por fecha, estadísticas al día y compresión.
- Caché para métricas y, si hace falta, una réplica de solo lectura.

¿Qué agregarías para producción?

- Secretos gestionados fuera del código y todo bajo HTTPS.
- Usuarios reales con contraseñas cifradas y tokens de refresco.
- Migraciones versionadas y copias de seguridad probadas.
- Monitoreo con logs, métricas y alertas.
- Integración continua con pruebas y escaneo de imágenes (ya incluida).
- Varias réplicas del backend tras un balanceador y base en alta disponibilidad.

Las contraseñas y el secreto de JWT traen un valor por defecto solo para que la demo funcione sin configurar nada. En un entorno real se definen en un archivo .env fuera del repositorio, y la API se niega a arrancar en modo producción si siguen con el valor por defecto. Existe además un informe de seguridad aparte en docs/Informe-Seguridad-Task-Manager.pdf.